Лабораторна робота № 7

Тема. Наскрізна автоматизація.

Мета: ознайомитися з наскрізною автоматизацією за допомогою Jenkins, Chef і AWS EC2.

Відповіді на контрольні запитання

1. За допомогою якої команди було створено віртуальну машину в Microsoft Azure?

Командою плагіна knife-azure з робочої станції Chef:

```
knife azure server create --azure-dns-name 'distechnodemo' --azure-vm-name
'dtserver02' \
  --azure-vm-size 'Small' -N DevOpsVMonAzure2 --azure-storage-account
'classicstorage9883' \
  --bootstrap-protocol 'cloud-api' --azure-source-image '<id образу CentOS
6.7>' \
  --azure-service-location 'Central US' --ssh-user 'dtechno' --ssh-password
'<пароль>' \
  -r 'role[v-tomcat]' --ssh-port 22
```

2. Який потік потрібен, щоб розгорнути PetClinic у веб-програмах Microsoft Azure (PaaS)?

PetClinic-Code (статичний аналіз SonarQube) → PetClinic (компіляція, модульні тести, пакування WAR) → PetClinic-Deploy-Azure (розгортання WAR у Azure App Service). PetClinic налаштовано як низхідне завдання для PetClinic-Code, а PetClinic-Deploy-Azure – як низхідне для PetClinic.

3. Як отримати додаткову інформацію про виконання випуску у VSTS?

Перейти до Releases, у стовпці Build & Release знайти потрібний випуск і двічі натиснути на нього (наприклад, Release-22): відкриються Summary, середовища, журнали кожного завдання, коміти, робочі елементи та тести.

4. Як створити середовище для розгортання пакетів у певному слоті веб-програми Azure?

У визначенні випуску натиснути «+ Add environment» → «Create new environment» (або «Clone selected environment»), задати умову запуску, хост виконання та назву (Dev/QA/UAT). У завданні Deploy Azure App Service

цього середовища відмітити «Deploy to Slot», вибрати групу ресурсів і потрібний слот, зберегти визначення.

5. Яка є альтернатива Microsoft Azure?

Amazon Web Services (EC2, Elastic Beanstalk), Google Cloud Platform (Compute Engine, App Engine), а також Oracle Cloud, IBM Cloud, DigitalOcean, Heroku. Для CI/CD замість VSTS – Jenkins, GitLab CI, GitHub Actions.

Порядок виконання роботи

Крок 1. Створення пакетних файлів.

Конвеєр Jenkins для PetClinic: статичний аналіз (SonarQube) → компіляція та модульні тести → пакування.

```
mvn clean sonar:sonar -Dsonar.host.url=http://localhost:9000  # PetClinic-
Code
mvn clean test  # PetClinic-Test
mvn package -DskipTests  # PetClinic (WAR)
```

```
[INFO] Tests run: 64, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building war:
/var/lib/jenkins/workspace/PetClinic/target/petclinic.war
[INFO] BUILD SUCCESS
```

На панелі Jenkins відображаються посилання на SonarQube-звіт проекту та тренд результатів модульних тестів; артефакт petclinic.war архівується (Archive the artifacts) для подальшого розгортання.

Крок 2. Налаштування доступу без пароля з Jenkins до робочої станції Chef.

На віртуальній машині Jenkins (від імені користувача jenkins):

```
sudo su - jenkins
ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
ls -l ~/.ssh
ssh-copy-id root@192.168.1.40  # IP робочої станції Chef
ssh root@192.168.1.40 'ifconfig | grep inet'
```

```
Number of key(s) added: 1
inet 192.168.1.40  netmask 255.255.255.0  broadcast 192.168.1.255
```

Помилку «Agent admitted failure to sign using the key» усунено командою ssh-add. Після цього з'єднання встановлюється без пароля, і Jenkins може виконувати команди knife на робочій станції Chef.

Крок 3. Створення екземпляра в AWS EC2 через Jenkins і Chef.

У завданні вільного стилю **PetClinic-Provision-EC2** додано крок «Execute shell»:

```
ssh root@192.168.1.40 "knife ec2 server create -I ami-0abcdef1234567890 -f
t2.micro \
  -N DevOpsVMonAWS --aws-access-key-id \"$AWS_ACCESS_KEY_ID\" \
  --aws-secret-access-key \"$AWS_SECRET_ACCESS_KEY\" -S book --identity-
file book.pem \
  --ssh-user ec2-user -r 'role[v-tomcat]'"
```

Ключі доступу AWS зберігаються в Jenkins Credentials і передаються як змінні середовища. Фрагмент Console Output:

```
Instance ID: i-0a1b2c3d4e5f67890
Flavor: t2.micro
Waiting for EC2 to create the instance........
Public IP Address: 3.120.xx.xx
Bootstrapping Chef on ec2-3-120-xx-xx...
Chef Infra Client finished, 12/15 resources updated
Finished: SUCCESS
```

Вузол DevOpsVMonAWS зареєстровано на сервері Chef (knife node list), на ньому встановлено Tomcat згідно з роллю v-tomcat.

Крок 4. Розгортання PetClinic в AWS Elastic Beanstalk (PaaS).

Потік: **PetClinic-Code → PetClinic → PetClinic-Deploy-ElasticBeanstalk**.

1. В AWS створено застосунок Elastic Beanstalk petclinic із середовищем petclinic-env (платформа Tomcat).

2. У Jenkins встановлено плагін *AWS Elastic Beanstalk Deployment*; у завданні PetClinic-Deploy-ElasticBeanstalk вказано регіон, Application name petclinic, Environment petclinic-env, Root object target/petclinic.war, Version label ${BUILD_NUMBER}, S3-bucket для версій.

3. PetClinic налаштовано як низхідне для PetClinic-Code, PetClinic-Deploy-ElasticBeanstalk – як низхідне для PetClinic.

```
Uploading file petclinic.war as
s3://elasticbeanstalk-eu-central-1-xxxx/petclinic/42.war
Creating application version 42 for application petclinic
Updating environment petclinic-env with version 42
Environment update finished: Status Ready, Health Green
Finished: SUCCESS
```

Застосунок доступний за URL середовища http://petclinic-env.eba-xxxx.eu-central-1.elasticbeanstalk.com.

Крок 5. Робота із середовищами Dev, QA, UAT.

У визначенні випуску VSTS для веб-програми Azure створено три середовища (Add environment → Create new environment / Clone):

Таблиця 7.1 – Налаштування середовищ випуску

Середовище	**Завдання**	**Слот**	**Умова запуску**
Dev	Deploy Azure App Service, Deploy to Slot	dev	автоматично після збірки
QA	Deploy Azure App Service, Deploy to Slot	qa	після успішного Dev
UAT	Deploy Azure App Service, Deploy to Slot	uat / stage	після QA, з попереднім схваленням (pre-deployment approval)

Для кожного середовища підписка AzureRM і назва служби програми використовуються ті самі; змінюється лише група ресурсів/слот. Для UAT налаштовано схвалення: у підсумку визначення випуску відображається попередження «Pre-deployment approval pending», і розгортання стартує лише після підтвердження відповідальною особою. Кожен слот – окрема жива веб-програма; результат розгортання перевірено за URL слота та через редактор Kudu (https://<app>-<slot>.scm.azurewebsites.net).

Висновки

У ході роботи побудовано наскрізну автоматизацію життєвого циклу застосунку PetClinic: статичний аналіз, компіляція, модульні тести та пакування WAR у Jenkins; безпарольний SSH-доступ з Jenkins до робочої станції Chef; створення екземпляра AWS EC2 і його налаштування через knife та роль Chef; розгортання в AWS Elastic Beanstalk як низхідне завдання конвеєра; а також розгортання в окремі слоти Azure App Service для середовищ Dev, QA і UAT зі схваленням. Наскрізна автоматизація скорочує ручні операції та забезпечує однаковий, відтворюваний шлях коду від коміту до робочого середовища.